HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Office for International Study Programs

Introduction to System on Chip
CO3143

Assignment Report

Pipelined Datapath

Lecturer: Phạm Kiều Nhật Anh
Class: CC02
Student: Nguyễn Viết Tấn Vương - 2353345

Ho Chi Minh City, 2025

Contents

1 Introduction **2**

2 DatapathPipelined.v **2**
2.1 Fetch stage (IF) 2
2.2 Decode stage (ID) 3
2.3 Execute stage (EX) 4
2.4 Memory stage (MEM) 7
2.5 Writeback stage (WB) 8

3 Test bench and Waveform **8**
3.1 Test bench code 8
3.2 Waveform 9

1 Introduction

The assignment focuses on the architectural of the design which turn a Multi-cycle module from the previous homework to a 5-stage Pipelined Datapath. The goal of this assignment is to implement a fully functional RV32IM pipelined processor that executes instructions through five stages: Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), and Writeback (WB).

2 DatapathPipelined.v

The module implements a 5-stage RISC-V pipelined processor. Unlike a single-cycle processor that finishes an instruction in one clock cycle, this design is divided into 5 stages which allows up to 5 instructions to be active simultaneously.

2.1 Fetch stage (IF)

- The `f_pc_current` holds the address of the current instruction.
- It sends the address `pc_to_imem` to the Instruction Memory.
- Next PC Logic: Normally the PC wil be `PC = PC + 4`. However, if a branch or jump is taken `pc_src == 1`, the PC is updated to `branch_target`.
- Pipeline Register (IF/ID): At the end of the cycle, the fetched instruction `inst_from_imem` and the PC are saved into the `if_id_...` registers and will be used for the next stage.

```
reg  [`REG_SIZE:0] f_pc_current;
wire [`REG_SIZE:0] f_pc_next;
wire [`REG_SIZE:0] f_inst;
assign f_pc_next = pc_src ? branch_target_pc : (f_pc_current + 4);

always @(posedge clk) begin
  if (rst) begin
    f_pc_current <= 32'd0;
  end
  else if (!stall_if) begin
    f_pc_current <= f_pc_next;
  end
end

assign pc_to_imem = f_pc_current;
assign f_inst = inst_from_imem;
always @(posedge clk) begin
    if (rst || pc_src) begin
        if_id_pc <= 0;
        if_id_inst <= 0;
    end
    else if (!stall_id) begin
        if_id_pc <= f_pc_current;
        if_id_inst <= f_inst;
    end
end
```

2.2 Decode stage (ID)

- Decoding: The instruction bits are split into `opcode`, `rd`, `rs1`, `rs2`,....
- Register Read: Reads values from the `RegFile` module using `rs1` and `rs2`.
- Immediate generation: Converts the immediate fields (I, S, B, U, J types) into 32-bit values `id_imm`.
- **H**azard Detection (Stall): If the previous instruction was a Load (`OpcodeLoad`) and it writes to a register that the current instruction needs, the pipeline stalls.
  - `stall_id = 1`: The PC and IF/ID registers keep their values.
  - `flush_ex = 1`: The next stage (EX) receives zeros which cancel the current instruction for one cycle.
- Pipeline Register (ID/EX): Passes the read values (`rs1_data`, `rs2_data`), immediates, control signals (`RegWrite`, `MemRead`,...) to the next stage.

```
wire [6:0] id_opcode = if_id_inst[6:0];
wire [4:0] id_rd     = if_id_inst[11:7];
wire [2:0] id_funct3 = if_id_inst[14:12];
wire [4:0] id_rs1    = if_id_inst[19:15];
wire [4:0] id_rs2    = if_id_inst[24:20];
wire [6:0] id_funct7 = if_id_inst[31:25];
wire [31:0] imm_i_sext = {{20{if_id_inst[31]}}, if_id_inst[31:20]};
wire [31:0] imm_s_sext = {{20{if_id_inst[31]}}, if_id_inst[31:25], if_id_inst
    [11:7]};
wire [31:0] imm_b_sext = {{19{if_id_inst[31]}}, if_id_inst[31], if_id_inst[7],
    if_id_inst[30:25], if_id_inst[11:8], 1'b0};
wire [31:0] imm_u_sext = {if_id_inst[31:12], 12'b0};
wire [31:0] imm_j_sext = {{11{if_id_inst[31]}}, if_id_inst[31], if_id_inst
    [19:12], if_id_inst[20], if_id_inst[30:21], 1'b0};
reg [31:0] id_imm;
always @(*) begin
    case (id_opcode)
        OpcodeStore:   id_imm = imm_s_sext;
        OpcodeBranch:  id_imm = imm_b_sext;
        OpcodeLui, OpcodeAuipc: id_imm = imm_u_sext;
        OpcodeJal:     id_imm = imm_j_sext;
        default:       id_imm = imm_i_sext;
    endcase
end

wire [31:0] id_rs1_data, id_rs2_data;
RegFile rf (
    .clk(clk), .rst(rst),
    .we(mem_wb_reg_we), .rd(mem_wb_rd), .rd_data(mem_wb_alu_res),
    .rs1(id_rs1), .rs1_data(id_rs1_data),
    .rs2(id_rs2), .rs2_data(id_rs2_data)
);
wire id_reg_we   = (id_opcode != OpcodeStore && id_opcode != OpcodeBranch);
wire id_mem_read = (id_opcode == OpcodeLoad);
wire id_mem_write= (id_opcode == OpcodeStore);
wire id_halt     = (id_opcode == OpcodeEnviron && if_id_inst[31:20] == 0);
assign stall_if = (id_ex_mem_read && (id_ex_rd != 0) && (id_ex_rd == id_rs1 ||
    id_ex_rd == id_rs2));
```

```
assign stall_id = stall_if;
assign flush_ex = stall_if || pc_src;

always @(posedge clk) begin
    if (rst || flush_ex) begin
        id_ex_pc <= 0;
        id_ex_rs1_data <= 0; id_ex_rs2_data <= 0;
        id_ex_imm <= 0;
        id_ex_rd <= 0; id_ex_rs1 <= 0; id_ex_rs2 <= 0;
        id_ex_opcode <= 0; id_ex_funct3 <= 0; id_ex_funct7 <= 0;
        id_ex_reg_we <= 0; id_ex_mem_read <= 0; id_ex_mem_write <= 0;
        id_ex_halt <= 0;
    end else begin
        id_ex_pc <= if_id_pc;
        id_ex_rs1_data <= id_rs1_data;
        id_ex_rs2_data <= id_rs2_data;
        id_ex_imm <= id_imm;
        id_ex_rd <= id_rd; id_ex_rs1 <= id_rs1; id_ex_rs2 <= id_rs2;
        id_ex_opcode <= id_opcode; id_ex_funct3 <= id_funct3; id_ex_funct7 <= id_funct7;
        id_ex_reg_we <= id_reg_we;
        id_ex_mem_read <= id_mem_read; id_ex_mem_write <= id_mem_write;
        id_ex_halt <= id_halt;
    end
end
```

2.3 Execute stage (EX)

- **F**orwarding Unit: This is critical. It checks if the data needed (`rs1` or `rs2`) is currently being written by a later stage (MEM or WB).
  - If the instruction in MEM stage writes to `rs1`, use `ex_mem_alu_res`.
  - If the instruction in WB stage writes to `rs1`, use `wb_final_data`.
  - Otherwise, use the value read from ID (`id_ex_rs1_val`).
- ALU Operations: The cla (Adder) or bitwise logic calculates the result.
- Branch Resolution: It compares the operands (`beq`, `blt`,...). If the condition is true, it sets `pc_src = 1`, which tells the IF stage to jump.
- Pipeline Register (EX/MEM): Passes the ALU result (`ex_alu_res`), the data to be stored (`ex_mem_wdata`), and destination register (`ex_mem_rd`) to the next stage.

```
reg [31:0] ex_op_a_val, ex_op_b_val;
always @(*) begin
    ex_op_a_val = id_ex_rs1_data;
    if (ex_mem_reg_we && (ex_mem_rd != 0) && (ex_mem_rd == id_ex_rs1)) begin
        ex_op_a_val = ex_mem_alu_res;
    end
    else if (mem_wb_reg_we && (mem_wb_rd != 0) && (mem_wb_rd == id_ex_rs1)) begin
        ex_op_a_val = mem_wb_alu_res;
    end
end
always @(*) begin
```

```
    ex_op_b_val = id_ex_rs2_data;
    if (ex_mem_reg_we && (ex_mem_rd != 0) && (ex_mem_rd == id_ex_rs2)) begin
        ex_op_b_val = ex_mem_alu_res;
    end
    else if (mem_wb_reg_we && (mem_wb_rd != 0) && (mem_wb_rd == id_ex_rs2)) begin
        ex_op_b_val = mem_wb_alu_res;
    end
end

  //operand
reg [31:0] alu_in_a;
reg [31:0] alu_in_b;
always @(*) begin
      if (id_ex_opcode == OpcodeAuipc) begin
          alu_in_a = id_ex_pc;
      end else begin
          alu_in_a = ex_op_a_val;
      end
      if (id_ex_opcode == OpcodeRegImm || id_ex_opcode == OpcodeLoad || id_ex_opcode == OpcodeStore || id_ex_opcode == OpcodeLui || id_ex_opcode == OpcodeAuipc || id_ex_opcode == OpcodeJalr) begin
         alu_in_b = id_ex_imm;
      end else begin
          alu_in_b = ex_op_b_val;
      end
end
// branch
wire is_beq  = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b000);
wire is_bne  = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b001);
wire is_blt  = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b100);
wire is_bge  = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b101);
wire is_bltu = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b110);
wire is_bgeu = (id_ex_opcode == OpcodeBranch) & (id_ex_funct3 == 3'b111);
wire cmp_eq  = (ex_op_a_val == ex_op_b_val);
wire cmp_lt  = ($signed(ex_op_a_val) < $signed(ex_op_b_val));
wire cmp_ltu = (ex_op_a_val < ex_op_b_val);

wire branch_cond = (is_beq & cmp_eq) | (is_bne & !cmp_eq) |
                   (is_blt & cmp_lt) | (is_bge & !cmp_lt) |
                   (is_bltu & cmp_ltu)| (is_bgeu & !cmp_ltu);

assign pc_src = branch_cond || (id_ex_opcode == OpcodeJal) || (id_ex_opcode == OpcodeJalr);
assign branch_target_pc = (id_ex_opcode == OpcodeJalr) ? ((ex_op_a_val + id_ex_imm) & ~1) : (id_ex_pc + id_ex_imm);

wire [31:0] cla_sum;
wire is_sub = (id_ex_opcode == OpcodeBranch) ||
              (id_ex_opcode == OpcodeRegReg && id_ex_funct7[5] == 1);

cla my_cla (
    .a(alu_in_a),
    .b(is_sub ? ~alu_in_b : alu_in_b),
    .cin(is_sub ? 1'b1 : 1'b0),
    .sum(cla_sum)
);

wire [31:0] div_rem, div_quot;
```

```
DividerUnsignedPipelined div (
    .clk(clk), .rst(rst), .stall(1'b0),
    .i_dividend(ex_op_a_val), .i_divisor(ex_op_b_val),
    .o_remainder(div_rem), .o_quotient(div_quot)
);

wire is_div_op = (id_ex_opcode == OpcodeRegReg) && (id_ex_funct7 == 7'b0000001);
reg [31:0] ex_alu_res;
always @(*) begin
    ex_alu_res = 0;
    if (id_ex_opcode == OpcodeLui) ex_alu_res = id_ex_imm;
    else if (id_ex_opcode == OpcodeJal || id_ex_opcode == OpcodeJalr) ex_alu_res = id_ex_pc + 4;
    else if (is_div_op) begin
         if (id_ex_funct3[2]) ex_alu_res = div_rem;
         else ex_alu_res = div_quot;
    end
    else if (id_ex_opcode == OpcodeLoad || id_ex_opcode == OpcodeStore || id_ex_opcode == OpcodeRegImm || id_ex_opcode == OpcodeRegReg || id_ex_opcode == OpcodeAuipc) begin
        case (id_ex_funct3)
            3'b000: ex_alu_res = cla_sum;
            3'b001: ex_alu_res = alu_in_a << alu_in_b[4:0];
            3'b010: ex_alu_res = ($signed(alu_in_a) < $signed(alu_in_b)) ? 1 : 0;
            3'b011: ex_alu_res = (alu_in_a < alu_in_b) ? 1 : 0;
            3'b100: ex_alu_res = alu_in_a ^ alu_in_b;
            3'b101: ex_alu_res = id_ex_funct7[5] ? ($signed(alu_in_a) >>> alu_in_b[4:0]) : (alu_in_a >> alu_in_b[4:0]);
            3'b110: ex_alu_res = alu_in_a | alu_in_b;
            3'b111: ex_alu_res = alu_in_a & alu_in_b;
        endcase
    end
end

always @(posedge clk) begin
    if (rst) begin
        ex_mem_pc <= 0; ex_mem_alu_res <= 0; ex_mem_wdata <= 0;
        ex_mem_rd <= 0; ex_mem_reg_we <= 0;
        ex_mem_mem_read <= 0; ex_mem_mem_write <= 0;
        ex_mem_opcode <= 0; ex_mem_funct3 <= 0;
        ex_mem_halt <= 0;
    end else begin
        ex_mem_pc <= id_ex_pc;
        ex_mem_alu_res <= ex_alu_res;
        ex_mem_wdata <= ex_op_b_val;
        ex_mem_rd <= id_ex_rd;
        ex_mem_reg_we <= id_ex_reg_we;
        ex_mem_mem_read <= id_ex_mem_read;
        ex_mem_mem_write <= id_ex_mem_write;
        ex_mem_opcode <= id_ex_opcode;
        ex_mem_funct3 <= id_ex_funct3;
        ex_mem_halt <= id_ex_halt;
    end
end
```

2.4 Memory stage (MEM)

- Uses `ex_mem_alu_res` as the memory address.
- Store: If it's a store instruction, it writes `ex_mem_wdata` to memory.
- Load: If it's a load instruction, it reads data from memory.
- Pipeline Register (MEM/WB): Passes the read data (`load_data`), ALU result, and destination register (`mem_wb_rd`) to the final stage.

```
always @(*) begin
    addr_to_dmem = ex_mem_alu_res;
    store_data_to_dmem = ex_mem_wdata;
    store_we_to_dmem = 0;

    if (ex_mem_mem_write) begin
        case (ex_mem_funct3)
            3'b000: begin
                store_data_to_dmem = ex_mem_wdata << (ex_mem_alu_res[1:0] * 8);
                store_we_to_dmem = 4'b0001 << ex_mem_alu_res[1:0];
            end
            3'b001: begin
                store_data_to_dmem = ex_mem_wdata << (ex_mem_alu_res[1] * 16);
                store_we_to_dmem = 4'b0011 << (ex_mem_alu_res[1] * 2);
            end
            default: begin
                store_data_to_dmem = ex_mem_wdata;
                store_we_to_dmem = 4'b1111;
            end
        endcase
    end
end

reg [31:0] mem_final_data;
always @(*) begin
    mem_final_data = ex_mem_alu_res;

    if (ex_mem_mem_read) begin
        case (ex_mem_funct3)
            3'b000: mem_final_data = {{24{load_data_from_dmem[7]}},
                load_data_from_dmem[7:0]};
            3'b001: mem_final_data = {{16{load_data_from_dmem[15]}},
                load_data_from_dmem[15:0]};
            3'b010: mem_final_data = load_data_from_dmem;
            3'b100: mem_final_data = {24'b0, load_data_from_dmem[7:0]};
            3'b101: mem_final_data = {16'b0, load_data_from_dmem[15:0]};
        endcase
    end
end

always @(posedge clk) begin
    if (rst) begin
        mem_wb_pc <= 0;
        mem_wb_alu_res <= 0;
        mem_wb_rd <= 0;
        mem_wb_reg_we <= 0;
        mem_wb_halt <= 0;
```

```
    end else begin
        mem_wb_pc <= ex_mem_pc;
        mem_wb_alu_res <= mem_final_data;
        mem_wb_rd <= ex_mem_rd;
        mem_wb_reg_we <= ex_mem_reg_we;
        mem_wb_halt <= ex_mem_halt;
    end
end
```

2.5 Writeback stage (WB)

- Selects the final data from Memory or ALU.
- This data (`wb_final_data`) is sent back to the `RegFile` to be written into register `rd`.

```
    always @(*) begin
    halt = mem_wb_halt;
    trace_writeback_pc = mem_wb_pc;
    trace_writeback_inst = 0;
end
```

3 Test bench and Waveform

3.1 Test bench code

```
module process_tb_assignment;
    reg clk;
    reg rst;
    wire halt;
    wire [31:0] trace_writeback_pc;
    wire [31:0] trace_writeback_inst;

    Processor uut (
        .clk(clk),
        .rst(rst),
        .halt(halt),
        .trace_writeback_pc(trace_writeback_pc),
        .trace_writeback_inst(trace_writeback_inst)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        #20;
        rst = 0;
    end

    always @(posedge clk) begin
        if (!rst) begin
            $display("%0t\t\t%h\t\t%h\t\t%b", $time, trace_writeback_pc,
                trace_writeback_inst, halt);
```

```
            if (halt) begin
                #50;
                $finish;
            end
        end
    end
    //  Timeout
    initial begin
        #50000;
        $finish;
    end
endmodule
```

3.2 Waveform

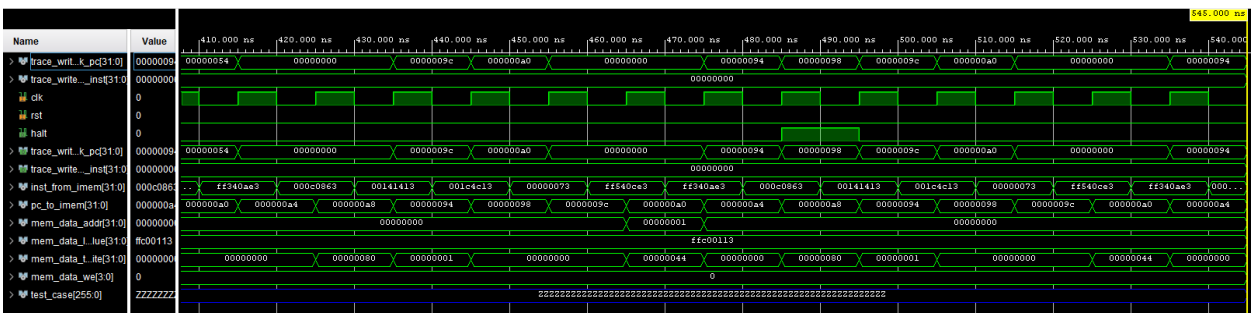

Figure 1: Simulation's waveform

- **Pipeline filling**:
  - At the beginning of the simulation (after Reset), instructions enter the pipeline sequential.
  - Cycle 1: Instruction 1 is fetched (pc_to_imem).
  - Cycle 2: Instruction 1 moves to Decode, instruction 2 is fetched.
  - Cycle 3: Instruction 1 moves to Execute, instruction 2 to Decode instruction 3 is fetched.
  - Cycle 5: The first instruction finally reaches the Writeback stage.
- **Forwarding**:
  - The waveform shows no stalls between these instructions. The ALU result is calculated correctly in the Execute stage and is going to the ALU inputs.
- **Load-Use hazard**:
  - The instruction stays in the Decode stage for one extra clock cycle.
  - The `stall_pipeline` signal goes High.
  - A NOP instruction is inserted into the Execute stage.
- **Branch flushing**:

  - The processor effectively "predicts not taken" by fetching the next sequential instruction (`PC+4`) while the branch is being calculated in the Execute stage.
  - When the branch condition is true (Taken), the `flush_decode` (or flush_execute) signal goes high (1).
  - The instructions currently in the Fetch and Decode stages are cleared and are replaced with NOPs.
  - The PC updates to the correct `branch_target`.

- **Divider**:
  - When the division reaches the Execute stage, the `stall_pipeline` (driven by `x_stall_div`) goes high (1) for 8 clock cycles.
  - The Fetch and Decode stages freeze.
  - The Memory stage receives NOPs during this wait.

- **Verification trace**:
  - The output signal `trace_writeback_pc` updates sequentially with the address of completed instructions.

— End —